



Este documento constitui **enunciado da Sprint 3** do **Trabalho Prático em Grupo de Programação Modular**. **Leia-o com atenção** e até ao fim antes de começarem a realizar as tarefas.

O bom andamento do trabalho prático em grupo **depende da realização correta e no prazo de cada**, portanto fique atento às suas obrigações. **Alunos que deixarem de realizar entregas em 2 ou mais etapas** estão sujeitos a serem **excluídos dos grupos** e, portanto, ficam com **nota 0 no trabalho como um todo**.

Título do projeto: *Xulambs Parking*

Na última etapa da implantação do sistema, surgem os requisitos finais para cobrança de uso dos estacionamento: Finalizada a *sprint* inicial, temos um aplicativo que consegue realizar as funções básicas pedidas pela *Xulambs Inc.*: cadastrar estacionamento e veículos; estacionar e liberar veículos; receber alguns relatórios simples. Podemos, então, avançar nos requisitos completos repassados pela empresa:

- A gerência resolveu fazer cobrança diferenciada pelos tamanhos dos veículos. Veículos pequenos pagarão 60% do valor original por hora. Veículos grandes pagarão 125%.
- Um veículo é cobrado de acordo com seu **plano de uso** dos estacionamento. Um veículo só pode usar um plano por vez e pode mudar de plano quando quiser.
- No plano **mensalista**, o veículo paga uma mensalidade fixa e não paga os usos do estacionamento por tempo. No plano **diarista**, com uma mensalidade menor, o veículo paga um valor máximo diário pelo tempo estacionado. Os veículos que não quiserem pagar por estes planos ficam cadastrados como **avulsos**, que pagam cada uso de vaga nas regras anteriores.
- Ao **fechar o mês**, um veículo terá adicionado ao valor total gasto o valor de sua possível mensalidade.
- Para refletir esta possibilidade, o aplicativo deve permitir a contratação de um plano por um veículo.
- Atendendo a pedidos da gerência, devem ser gerados os relatórios:
 - Dados de um veículo específico, com sua placa, valor total e detalhamento de cada uso;
 - Média de gasto dos veículos cadastrados no aplicativo;
 - Lista de veículos com gasto maior do que a média;
 - Exibir os 3, 5 ou 10 veículos com o maior gasto total;
 - Lista de veículos que estacionaram em um dia determinado pela gerência.
- Os requisitos obrigam a modificação das classes *UseDeVaga* e *Veículo* e a criação os planos de uso.
- Todas as classes devem ser devidamente documentadas. Além disso, todas as situações de quebra de contrato em seus usos devem lançar exceções correspondentes ao problema acontecido.
- Finalmente, para demonstração do sistema, será preciso criar um gerador de dados aleatórios que obedeça às características:
 - Gerar pelo menos 2 veículos de cada tipo; (sugestão: 32 veículos)
 - Gerar pelo menos 1 veículo com cada plano (sugestão: total / 3)
 - Gerar em média 10 usos por veículo (sugestão: usar os novos métodos “estacionar” e liberar do veículo, com datas, gerando entradas sequenciais).

Esta função fica num método *static* de uma classe *GeradorDados*, que tem esta responsabilidade.

Estes requisitos resultaram na **atualização do diagrama** que pode ser visto no final deste documento. Ele deve ser seguido para que o aplicativo funcione bem ao final da implementação.

Preparação:

- Os códigos das tarefas anteriores foram comentados pelo professor **nas suas ramificações** do repositório em <https://classroom.github.com/a/FuBgBVR>.
- Foi criada uma ramificação de nome *joaoCaram/versao2*, contendo a versão completa do código do professor. Usem essa versão para implementarem as novas classes de vocês.
- Lembre-se número designado a você em aula como **número de aluno** (de 1 a 5) para novas as tarefas de implementação.

Tarefa 5 (tarefa individual):

- De acordo com **seu número de aluno**, realize as **implementações** a seguir:
- **Crie uma ramificação**, a partir da *main* unificada, chamada **seuNome/Sprint3** (por exemplo, juçaraMarçal/Sprint3);
- De acordo com **seu número de aluno**, realize as **implementações** a seguir:
 1. Gerador de dados aleatórios.
 2. Versão do aplicativo chamando o gerador de dados, permitindo contratar planos para veículos e com os relatórios;
 3. Adaptações da classe Veículo;
 4. Adaptações da classe UsoDeVaga e criação dos métodos sobrecarregados no Estacionamento;
 5. Atualização do enumerador e criação dos planos de uso

Tarefa 6 (tarefa de grupo):

Realizar o último merge do repositório a tempo de apresentar o trabalho para o professor. Este merge deverá, obrigatoriamente, utilizar o aplicativo disponibilizado. Fique atento às instruções nas aulas para a entrega e a apresentação do trabalho.

Prazos:

- A apresentação está prevista para o dia 01/07.

Instruções e observações:

- O projeto deve estar **obrigatoriamente hospedado** na tarefa do GitHub Classroom: <https://classroom.github.com/a/FuBgBVR>;
- **Atenção para as datas** das tarefas. As **avaliações** das atividades serão feitas **sempre** com o **último commit / pull request constante daquela data**.
- **Atenção para a política de uso de agentes de código e “inteligência artificial”** nas tarefas. Seu uso é **permitido, sob duas condições**:
 - O aluno deve **declarar**, no **cabeçalho** das classes, que **foi utilizado um agente de código**;
 - O aluno deve realizar **pelo menos dois commits**: o **primeiro** com o **código gerado automaticamente** e um **segundo** com as **revisões e melhorias realizadas** pelo aluno – as quais devem estar comentadas.
 - Em caso de **códigos com indícios de uso de agentes sem cumprir as condições acima**, a atividade terá sua **nota zerada, sem direito a recurso**.
- **A nota do aluno** em cada etapa é **vinculada às suas entregas individuais** e à nota de grupo. Alunos que **não cumprirem sua tarefa individual** ficam **sem nota**, mesmo que o grupo tenha cumprido as tarefas coletivas.

